

Aufgabenstellung für Assignment zu „Ausgewählte  
Aspekte des Software-Engineering I“(SE3AA1)

michael.decker@dhbw-karlsruhe.de

Version vom 30. April 2026

# Überblick

Die erste Lehrveranstaltung „Ausgewählte Aspekte des Software Engineerings 2“ im Modul „Software Engineering III“ wurde über eine Klausur abgeprüft. Die zweite Lehrveranstaltung „Ausgewählte Aspekte des Software Engineerings 1“<sup>1</sup> hingegen wird über ein Assignment abgeprüft, es handelt sich also um eine sog. *Kombinierte Modulprüfung*. Im vorliegenden Dokument findet sich hierfür die Aufgabenstellung.

Bitte beachten Sie, dass Gruppenarbeit bei Assignments nicht zulässig ist. Weiter wird es auch keine benoteten (Abschluss-)Präsentationen geben.

**In Absprache mit dem Dozenten kann auch eine eigene Aufgabenstellung gewählt werden.**

---

<sup>1</sup>„Ausgewählte Aspekte 2“ findet im 5. Semester **vor** „Ausgewählte Aspekte 1“ statt.

## Aufgabenstellung

Es ist mit Java, Maven und *Spring Boot* eine Web-Anwendung zu erstellen, mit der beliebige Daten in einer in der Vorlesung vorgestellten Datenbank gespeichert werden. Beispiele für Daten, die mit der Anwendung verwaltet werden können, sind z.B.:

- Notizen
- Kalendereinträge
- Adressbuchkontakte
- Quellen für eine wissenschaftliche Arbeit
- Aufgaben (To-Dos)
- Persönliches Inventar (z.B. Briefmarkensammlung, Bücher, Comics, T-Shirt, Tonträger, Brettspiele... etwas, das Sie sammeln).
- Link/URL-Sammlung
- Fahrtenbuch
- Mini-CMS<sup>2</sup> z.B. für eine FAQ-Webseite.

Für die Persistierung der Daten ist eine der folgenden in der Vorlesung vorgestellten Persistenz-APIs, die von *Spring Boot* unterstützt wird, zu verwenden:

- H2 mit JPA
- H2 mit *Spring Data JPA*
- Cassandra mit *Spring Data*
- MongoDB mit *Spring Data*
- Neo4j mit *Spring Data*

Die Verwendung der in der Vorlesung aus didaktischen Gründen vorgestellten API `JdbcTemplate` ist nicht zulässig. Für die verwalteten Daten sind die vier CRUD-Operationen<sup>3</sup> zu unterstützen, wobei mindestens zwei Tabellen verwendet werden (bei MongoDB: mindestens zwei Collections; bei Neo4j: zwei Knotentypen). Wenn Sie für Ihren Anwendungsfall nur eine Tabelle benötigen, dann

---

<sup>2</sup>Content Management System, siehe auch <https://bit.ly/4mJL7Rr>

<sup>3</sup>Create, Read, Uppdate, Deleate

kann die zweite Tabelle für Authentifizierungsinformationen genutzt werden. Wenn die zwei Tabellen für die Anwendungsdaten benötigt werden, dann ist die Implementierung einer Authentifizierung nicht erforderlich. Die Persistenz soll die Daten tatsächlich in eine Datei auf Festplatte/SSD schreiben<sup>4</sup>, so dass Datenänderungen durch einen Neustart der Anwendung nicht verloren gehen. Wenn Sie eine andere Datenbank als H2<sup>5</sup> verwenden, dann muss das Repository eine Datei `docker-compose.yml` für einen entsprechend konfigurierten Docker-Container enthalten<sup>6</sup>.

Das Datenmodell ist als Diagramm darzustellen (entweder als UML Klassendiagramm oder ER-Diagramm nach Chen) und an einem der Vorlesungstermine zu präsentieren. Diese Präsentation wird nicht bewertet. Das Diagramm mit dem Datenmodell muss als PDF-, SVG- oder PNG-Datei im Quellcode-Repository auf oberster Ebene enthalten sein.

Da es sich bei der Anwendung um eine Web-Anwendung handeln soll, ist die Nutzeroberfläche basierend auf HTML zu entwickeln. Die HTML-Seiten können hierzu entweder mit der Template-Engine *Thymeleaf* erzeugt werden, oder statische HTML-Seiten können mit JavaScript auf eine mit `RestController` implementierte REST-API auf das Backend zugreifen. Diese beiden Ansätze können auch gemischt werden. Wenn Sie möchten, dann können Sie auch ein CSS-Framework wie Bootstrap verwenden. Eine rein funktionale Nutzeroberfläche (wie von den Beispielanwendungen aus der Vorlesung) ist ausreichend.

Zusätzlich zum Quellcode der Anwendung (Upload im Moodle-Raum als ZIP-Datei und Git-Repository) ist noch ein kurzes Video zu erstellen, das einen Überblick über die grundlegende Bedienung der Anwendung beinhaltet. Sie finden genaue Anforderungen an dieses Video in einem Kapitel weiter hinten im vorliegenden Dokument.

Wenn Sie eine Anwendung für den privaten Gebrauch nach Ende Ihres Studiums an der DHBW erstellen möchten, dann kann es ausreichend sein, ein *Minimum Viable Product (MVP)* zu erstellen; in diesem Fall soll die Nutzeroberfläche aber keine Elemente wie Menüpunkte enthalten, die noch keine Funktionen aufweisen, also z.B. ständig ausgegraut sind.

---

<sup>4</sup>sonst wäre es keine echte Persistenz; H2 unterstützt aber auch eine „Speicherung“ der Daten nur im RAM (In-Memory-Modus), siehe z.B. <https://bit.ly/4sKdqR3>

<sup>5</sup>H2 ist eine kleine relationale Datenbank, die im selben Java-Prozess wie die Spring-Boot-Anwendung laufen kann

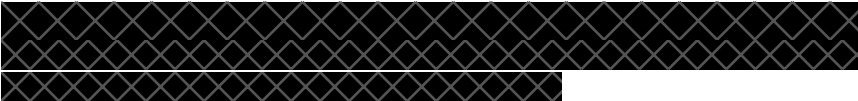
<sup>6</sup>als Vorlage können Sie hierfür die `docker-compose.yml` der entsprechenden Beispiel-Anwendung aus der Vorlesung verwenden; bei Cassandra und MongoDB ist jeweils eine Datenbankinstanz ausreichend

## Bewertungskriterien

Die von Ihnen erstellte Anwendung wird nach den folgenden Kriterien bewertet:

- Funktionale Korrektheit und Robustheit der Anwendung.
- Bei der Web-Oberfläche wird nicht ein „schickes“ Aussehen bewertet, es kann sich auch um eine schlichte Oberfläche handeln; die Oberfläche muss aber ein konsistentes Aussehen aufweisen und gut bedienbar sein, also z.B.:
  - Texte müssen gut lesbar sein, haben also einen guten Kontrast zum Hintergrund und setzen nicht direkt auf den (Viewport-)Rand auf.
  - Konsistente Zahlen- und Datumsformate passend zur Sprache der Anwendung (z.B. „.“ nicht als Dezimaltrenner, wenn die Anwendung auf Deutsch ist); die Anwendung muss nur eine Sprache (vorzugsweise Deutsch, evtl. auch Englisch) unterstützen.
  - Sicherheitsabfrage vor Löschoperationen.
- Sinnvolle Behandlung von ungültigen Eingaben.
- Intuitive Bedienung.

Anforderungen an den Quellcode:

- 
- Bei Verwendung von H2 muss das Datenmodell normalisiert sein (Daten sind ggf. auf mehreren Tabellen aufgeteilt, damit es nicht zu redundanten Daten in einer Tabelle kommt).
- Beim Exception-Handling dürfen die `catch`-Blöcke nicht leer sein, es muss zumindest eine Warnung auf den Logger geschrieben werden oder – wenn der Fehler für den Nutzer relevant ist – in einem Dialog angezeigt werden.
- `import`-Anweisungen müssen konkrete Klassen importieren, Wildcard-Imports wie z.B. `import java.util.*` sind nicht zulässig.
- Der Quellcode sollte mindestens einige Kommentare enthalten; Sie müssen aber nicht unbedingt jede einzelne Methode oder Klasse mit einem Kommentar versehen. Kommentare für eine ganze Methoden, Klassen oder Objektvariablen sollten als JavaDoc-Kommentar mit `/** . . . */` geschrieben werden.

- Es müssen regelmäßig Commits in das bei der Anmeldung angegebene Git-Repo auf GitHub.com oder dem DHBW-eigenen GitLab-Server gepusht werden. Aus prüfungsrechtlichen Gründen muss aber der komplette Quellcode auch in Form einer ZIP-Datei ins Moodle hochgeladen werden.
- Wählen Sie gut verständliche Bezeichner, z.B. Namen für Klassen oder Variablen.
- Beachten Sie die gängigen Namenskonventionen für Java-Quellcode, z.B. lowerCamelCase-Schreibweise für lokale Variablen oder UpperCamelCase für Klassennamen<sup>7</sup>
- Quellcodezeilen dürfen nicht länger als 120 Zeichen sein.
- Methoden dürfen nicht mehr als 50 Zeilen umfassen.
- Verwenden Sie einheitliche Einrückungen und Leerzeilen.
- Vermeiden Sie „Duplicate Code“.
- Der Quellcode sollte keine auskommentierten Quellcodezeilen enthalten (sog. Code-Leichen), da Sie später vielleicht doch wieder benötigte Quellcodefragmente über Git wiederherstellen können.
- Es darf keine lokalen oder globalen Variablen geben, die nicht verwendet werden oder die nur befüllt, aber nicht ausgelesen werden.
- Es darf keine `import`-Anweisungen geben, die nicht benötigt werden.
- Die Klassen sind auf Packages zu verteilen, wobei ein Package mindestens zwei Klassen enthalten muss.
- Verwenden Sie die Annotation `@Override` beim Überschreiben von Interface-Methoden und aus Oberklassen geerbten Methoden, um sich bewusst zu machen, dass eine Methode überschrieben wird. Außerdem werden Sie dann vom Compiler darauf aufmerksam gemacht, wenn Sie den Namen der zu überschreibenden Methode falsch geschrieben haben oder die Parameter nicht passen.
- Es dürfen keine als „deprecated“ gekennzeichneten Methoden und Klassen verwendet werden.
- Für einfache Datentransportklassen sind Records zu verwenden.
- Beachten Sie auch die in der Vorlesung behandelten *Best Practices*.
- Lombok<sup>8</sup> darf nicht verwendet werden.

---

<sup>7</sup><https://bit.ly/42sRf76>

<sup>8</sup><https://projectlombok.org/>

Anforderungen für die Erleichterung der Bewertung:

- Die Anwendung muss sich durch Eingabe des folgenden Befehls im Hauptverzeichnis über den Maven-Wrapper starten lassen:

```
mvnw spring-boot:run
```

- Die App oder eine Einstiegs-Seite der App muss auf dem Default-Port 8080 erreichbar sein, also bei lokalem Aufruf unter der folgenden URL:

```
http://localhost:8080
```

- Bei Verwendung von H2 als Datenbank muss die H2-Konsole für die Datenbank aktiviert und bei lokalem Aufruf unter der folgenden URL aufrufbar sein:

```
http://localhost:8080/h2-console/
```

## Anforderungen an Video

Stellen Sie sich vor, dass das Video ein kurzes Tutorial ist, mit dem ein neuer Endanwender Ihrer Anwendung einen Einstieg in die Bedienung der Anwendung bekommt.

- Bildschirm bei Bedienung der Anwendung abfilmen und auf Tonspur mit Mikrofon erklären, was gerade gemacht wird.
- Laufzeit: zwei bis drei Minuten.
- Es darf nicht geschnitten oder bearbeitet sein.
- Das Video muss sich mit dem „VLC Media Player“ unter Windows abspielen lassen.
- Das Video ist ebenfalls im Moodle-Raum über das entsprechende Abgabelement hochzuladen (eine Bereitstellung über YouTube ist aus prüfungsrechtlichen Gründen nicht ausreichend).
- Anforderungen an die Tonspur: Keine Hintergrundmusik, mit Headset aufgenommen (bei Aufnahme Sprache mit in Laptop eingebautem Mikrofon ist die Qualität oft zu schlecht), keine Hintergrundgeräusche (z.B. Straßenlärm).
- Die Auflösung des Videos muss so hoch sein, dass die Texte der Anwendung gut zu lesen ist.
- Die abgefilmte Anwendung muss sich in einem Zustand befinden, in dem alle Texte der Anwendung gut lesbar sind.
- Das Video darf nicht abrupt enden.
- Da es sich um ein Video für die Endanwender Ihrer Anwendung handeln soll, sind keine technischen Details wie Quellcode zu zeigen.



## Anmeldung

Bitte melden Sie Ihr Thema bis spätestens 31. Mai 2026 (So) um 23:59 Uhr über das entsprechende Element im letzten Abschnitt des Moodle-Raums an (bei einer verspäteten Anmeldung gibt es Punkte-Abzug bei der Bewertung). Bei der Anmeldung sind folgende Angaben zu machen:

- Titel der Anwendung, z.B. „Bierdeckelverwaltung“.
- Die Namen der beiden Tabellen/Entities, die in der Datenbank abgelegt werden (für MongoDB: Namen der Collections; für Neo4j: Namen der Knotentypen).
- Vor- und Nachname
- Email-Adresse
- URL auf Git-Repo (leer lassen, wenn Sie ein Repo auf dem DHBW-internen GitLab-Server haben möchten).
- Kommentar/Anmerkung (optional)
- Persistenztechnologie (Auswahl aus Dropdown-Menü).

Wenn Sie ein Git-Repo auf GitHub.com (gehört inzwischen zu Microsoft) verwenden möchten, dann müssen Sie dies selbst anlegen und den GitHub-Nutzer **MDecker-MobileComputing** als „Collaborator“ zum Repo einladen<sup>9</sup>. Bitte verwenden Sie nur ein **privates** Repo; am besten wählen Sie auf der Seite zum Anlegen eines neuen Repos<sup>10</sup> unter „Choose Visibility“ gleich „Private“ aus<sup>11</sup>. Ein öffentliches Repo kann aber auch nachträglich von „Public“ auf „Private“ umgeschaltet werden<sup>12</sup>.

Wenn Sie kein Git-Repo auf GitHub.com verwenden möchten, dann kann der Dozent für Sie ein Git-Repo auf dem DHBW-internen GitLab-Server anlegen. In diesem Fall lassen Sie das Feld mit der URL für das GitHub-Repo bitte leer und schreiben eine entsprechende Anmerkung in das Kommentarfeld. Dieser GitLab-Server ist außerhalb dem Netzwerk der DHBW aber nur über eine VPN-Verbindung zur DHBW erreichbar. Loggen Sie sich auf dem GitLab-Server mindestens einmal ein, damit der Server vom zentralen Verzeichnisdienst Ihren Namen holt und diesen speichert. Sie erreichen diesen GitLab-Server unter der folgenden URL: <https://git.dh-karlsruhe.de>

---

<sup>9</sup><https://el-decker.de/github-nutzer-einladen/>

<sup>10</sup><https://github.com/new>

<sup>11</sup>[https://el-decker.de/GitHub\\_NeuesRepo\\_VisibilityPrivate.png](https://el-decker.de/GitHub_NeuesRepo_VisibilityPrivate.png)

<sup>12</sup>[https://el-decker.de/GitHub\\_RepoSettings\\_ChangeVisibility.png](https://el-decker.de/GitHub_RepoSettings_ChangeVisibility.png)

## Präsentationen

Es gibt zwei Arten von Präsentationen während der Vorlesung, die beide **nicht** benotet werden:

- Vorstellung Datenmodell (Pflicht)
- Vorstellung gesamte App (Freiwillig)

Jeder Kursteilnehmer muss sein Datenmodell in einer Kurzpräsentation mit etwa 3 Minuten Dauer vorstellen. Hierfür muss keine Präsentationsdatei erstellt werden, das Diagramm (UML Klassendiagramm oder ER-Diagramm nach Chen) ist ausreichend.

Gegen Ende der Vorlesungszeit wird es mindestens einen Vorlesungstermin geben, an dem Sie freiwillig die vollständige Anwendung präsentieren können, um vor der Abgabe Feedback einzuholen. Diese Präsentation kann wegen der Prüfung als Assignment aber nicht bewertet werden, hilft Ihnen aber, Ihre Anwendung vor der Abgabe zu verbessern. Wenn Sie von dieser Möglichkeit Gebrauch machen wollen, dann schreiben Sie bitte rechtzeitig eine Email an den Dozenten (siehe Titelseite für Email-Adresse). Sollten Sie in der Klausur „Ausgewählte Aspekte des Software Engineerings 2“ eine Bewertung schlechter als 4.0 erzielt haben, dann wird dringend empfohlen, von dieser Möglichkeit Gebrauch zu machen.

Schreiben Sie eine Email an den Dozenten, um einen Termin für eine der beiden Präsentationen zu vereinbaren.